Murray State University Honors College

HONORS THESIS

Certificate of Approval

Using Machine Learning to Optimize The Board Game Can't Stop

J. Cameron Carter
May 2026

Approved to fulfill the
requirements of HON 437

Dr. Christopher Mecklin, Advisor
Mathematics and Statistics

Approved to fulfill the
Honors Thesis requirement
of the Murray State Honors
Diploma

Dr. Jessica Naber, Executive Director
Honors College

Examination Approval Page

Author: J. Cameron Carter

Project Title: Using Machine Learning to Optimize The Board Game Can't Stop

Department: Mathematics and Statistics

Date of Defense: April 22nd, 2026

Approval by Examining Committee:

Dr. Christopher Mecklin, Advisor	Date
Dr. Md. Mahadi Hasan, Committee Member	Date
Dr. Manoj Pathak, Committee Member	Date

Using Machine Learning to Optimize The Board Game Can't Stop

Submitted in partial fulfillment
of the requirements
for the Murray State University Honors Diploma

J. Cameron Carter

May 2026

ACKNOWLEDGEMENTS

I would like to thank the entire Murray State Department of Mathematics and Statistics, my advisor Dr. Mecklin, and my supportive friends and family. A very special thank you to my mom, who is the backbone of this project's grammar and spelling. This project would not have been possible without all of their wonderful support and encouragement.

ABSTRACT

Can't Stop is a jeopardy dice game invented by Sid Jackson and first published in 1980. It has players rolling four dice to make pairs that advance markers up a board, with the first player to complete three columns winning the game. One of the most important decisions each turn is deciding whether to stop rolling the dice to keep your progress, or risk failing the next roll and losing your progress. Different strategies have been developed on when is the optimal time to stop, using a variety of statistical and mathematical approaches. However, one approach that has not seen significant development is neural networks, a machine learning method. Through experimentation and computer simulations, the goal is to see if using the reinforcement learning algorithm Deep Q-Learning (a type of neural network) will produce a better strategy for when to stop rolling than existing strategies, such as Rule of 28 (Keller 1986) or Generalized Heuristic (Glenn and Aloi 2009).

Contents

1 Introduction **1**

1.1 Can't Stop Description 1

1.2 Game Materials 3

1.3 Rules 3

2 Literature Review **5**

3 Methods **8**

3.1 Q-Learning 8

3.2 Neural Network 9

3.3 Hardware and Software 10

3.4 Strategies 10

3.4.1 Random Player 11

3.4.2 Cautious Player 11

3.4.3 Aggressive Player 12

3.4.4 Rule of 28 Player 12

3.5 DQN Structure 13

3.6 Training 15

4 Analysis and Results **17**

4.1 Introduction 17

4.2 Predicted vs Actual Q-values 17

4.2.1 Observations and Analysis 25

4.2.2 Conclusion 27

4.3 Opponent Win Rates 28

4.3.1 Observations and Analysis 31

4.3.2 Conclusion 33

4.4 Feature Importance 34
4.4.1 Observations and Analysis 36
4.4.2 Conclusion 37

5 Conclusion **38**

References **40**

A Appendix: Odds Table **42**

B Appendix: Python Code & Github **44**

List of Tables

1 Rule of 28 Scoring (Keller 1986) 13
2 Average Predicted and Actual Q-values 18
3 Parameter Values 25
4 Permutation Feature Importance 34

List of Figures

1 Sample Can't Stop game in progress 1
2 DQN Model Architecture 15
3 Runs 1-3 Rewards vs Predicted Q-values (smoothed) 19
4 Runs 4-6 Rewards vs Predicted Q-values (smoothed) 20
5 Runs 7-9 Rewards vs Predicted Q-values (smoothed) 21
6 Runs 10-12 Rewards vs Predicted Q-values (smoothed) 22
7 Runs 13-15 Rewards vs Predicted Q-values (smoothed) 23
8 Runs 16-18 Rewards vs Predicted Q-values (smoothed) 24
9 Overall Win Rate per Training Session 28

10 Versus Random Win Rate per Training Session 28
11 Versus Ro28 and Smart Ro28 Win Rate per Training Session 29
12 Versus Cautious Players Win Rate per Training Session 30
13 Versus Aggressive Players Win Rate per Training Session 31
14 Can't Stop Odds (Tate and Renner 2007) 42
15 Can't Stop Odds (Tate and Renner 2007) 43

1 Introduction

1.1 Can't Stop Description

Can't Stop is a board game developed by Sid Sackson and published by Parker Brothers in 1980. It is a two to four player jeopardy dice game where each player tries to get to the top of three columns first. We define a jeopardy dice game as a game that uses one or more dice as a random device, where the main decision is whether or not to jeopardize previous gains by rolling dice for possible greater gains with a risk of losing all gains from that turn.

In Can't Stop, during a player's turn they roll 4 six-sided dice, which will allow the player to either make progress (greater gains) towards the top of a column or reset their progress (jeopardize previous gains) and end the turn.

Other games in the jeopardy dice game genre include Pig, Cosmic Wimpout, Twenty-One, and Macao.

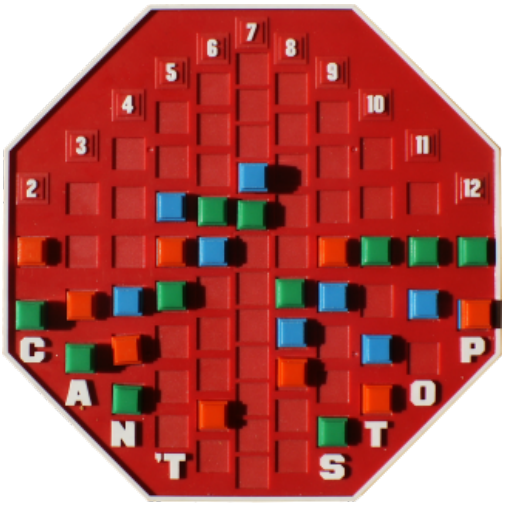

Figure 1: Sample Can't Stop game in progress

[1]

Informally, we define a game as competitive rule-based play involving multiple players. The players take turns making progress towards a goal, and the first player to reach this

[1]Figure 1 Retrieved from `https://en.wikipedia.org/wiki/Can%27t_Stop_(board_game)`

goal is deemed the winner. A mathematical definition of a game is the specification of a set of players, rules for playing the game, and an assignment of payoffs or utilities to all possible endings resulting from the various actions of the players. Rules specify the order and consequence of moves for the various players, how the game ends, and who gets what under various endings (Packel 2006). We can further define a player and payoff as someone who makes choices and receives payoffs, and a positive or negative number assigned to each possible game end respectively (Rapoport 1966). While there are different types of games based on the overall payoff, we will only consider zero-sum games, a game where the players' payoffs sum to zero. Can't Stop is a zero-sum game.

A zero-sum game is a game of competitive interactions where one player's gain is another player's loss. These types of games can be solved, which means we can determine the outcome or possible outcomes of each game, given certain assumptions about the players. These solutions develop into a strategy, where the answer to a player's choices are specified in advance. The optimal play, also referred to as a saddle point or minimax, is the move that maximizes your payoff while minimizing your opponent's payoff based on the opponent's move choice. For each move you can make, you assume your opponent picks your worst-case scenario loss. From all the possible worst cases, pick the move that with the smallest loss. This process is minimizing your maximum losses. The other player will do the opposite, trying to maximize their minimum gains. The move choices of each player will result in optimal play. Assuming both players will act rationally and an optimal play exists, then both players will always have to pick the same move, since that move's payoff is the best they can do knowing the other will act rationally. A dominant strategy is mathematically unexploitable by other players, ensuring positive payoff regardless of your opponent's actions. Regardless of the opponent's move choice, you will always be maximizing your gains. Exploitable play uses opponents' mistakes and strategies to maximize your own payoff, by targeting an identified weakness.

As an example, here is how these different strategies would look like in the game Rock-

Paper-Scissors. The optimal play is to pick one of the three options randomly. Since you assume your opponent is playing rationally, the random choice maximizes your winning chance without giving your opponent an advantage, minimizing their winning chance. A dominant strategy would be if one player could use a hypothetical fourth move, such as "dynamite", that always wins. This move would be a dominant strategy for that player. The exploitable play would be a scenario where you notice the opponent always picks rock on the second round, so you exploit this behavior by always picking paper on the second round. However, while adopting an exploitive strategy can increase your gains, it exposes you to being re-exploited by your opponent. In the Rock-Paper-Scissors situation, if you start always picking paper, a clever opponent will start picking scissors.

Another example of exploitive play is in the card game poker. In poker players often can take advantage of their opponent's mistakes, like never bluffing or bluffing too often. This would not be an optimal play because the player's strategy is not assuming the opponent is making their best decisions. Since the opponent is not making optimal decisions the player can exploit their opponent.

1.2 Game Materials

The board consists of 11 columns, numbered 2 through 12, with lengths 3, 5, 7, 9, 11, 13, 11, 9, 7, 5, and 3 respectively. The game uses 4 six-sided dice, four colored sets of 11 markers, and three neutral-colored markers (runners).

1.3 Rules

On a player's turn, they roll all 4 six-sided dice at once. They form two pairs of numbers with the dice that they have rolled. The sums of these pairs determine the column(s) in which the player must introduce a runner. You may have only a total of 3 runners active in any given turn.

The player rerolls all 4 dice and again forms two dice pairs. After doing so, they again

introduce or move one or two runners.

Turns continue like this until the player stops voluntarily or is forced to stop because they cannot introduce or move a runner.

If a player stops voluntarily, they replace each of their runners with one of their markers on the game board. If during a later turn, the player introduces a runner in a column that already has a marker, they place the runner on the square above the one occupied by their marker.

If a player is forced to stop, they remove the runners from the board, losing all progress achieved that turn, and their turn ends. This is referred to as a bust, or busting.

If a player's runner reaches the top of the column, and then they choose to stop, the player wins that column. The player to win 3 columns first wins the game.

As an example, consider this scenario:

At the start of a player's turn, they roll the 4 dice and get 2,3,4,5. Then the possible pairs are (2+3,4+5), (2+4,3+5), or (2+5,3+4), giving the player either (5,9), (6,8), or (7,7). If the player chooses the first pair, they place a runner on the bottom square of the 6 column and another runner at one the bottom square of the 9 column. If the player chooses the second pair, they place a runner on the bottom square of the 6 column and another runner on the bottom square of the 8 column. If the player chooses the third pair, they place a runner on the bottom square of the 7 column and then move it one square upwards. Each roll will result in at most three distinct pairs. A roll can return less than three possible pairs, such as 1,1,5,6 which gives (2,11) or (7,6), 3,3,3,5 which gives (6,8), or 4,4,4,4 which gives (8).

2 Literature Review

Research on finding optimal strategies and solving Can't Stop has been ongoing since the game's release in 1980. A main focus is developing a heuristic strategy or "rule of thumb", defined as a strategy that is easy and quick to execute for any player while gaining a meaningful advantage over other players. The earliest heuristic strategy is the Rule of 28 (Keller 1986). This strategy creates an easy-to-remember system of point scoring based on column and move choices, where the player stops upon reaching a score threshold of 28. The points for each column are based on the probability of getting that column in a roll, with more likely numbers being assigned less points, and unlikely numbers assigned high points. The Rule of 28 (Ro28) serves as a useful benchmark for developing other strategies.

A common approach to finding strategies is using computer simulations to approximate or solve a game. Complicated games like Can't Stop are difficult to solve due to the computation time, so breaking the game down into simple, solitaire versions or "toy" games, which are solvable can provide useful insight into the full game. The Retrograde Approximation Algorithms have solved simple versions of one-player games, up to 5-sided dice with the shortest column of length 2. 2-player games were solved up to 4-sided dice with a shortest column of 1 (Fang, Glenn, & Kruskal 2008). These algorithms combine Newton's method and retrograde analysis to solve the game by modeling it as a finite, cyclic graph which can be used to show the existence of unique optimal solutions. The algorithms are iterated over the graph representations until the algorithm converges to a value after enough time. From the value they get the game theoretic optimal play. From these solutions, patterns can be found to approximate optimal strategies for the full game.

The Rule of 28 heuristic was generalized, so it can be used for the toy games (Glenn & Aloi 2009). However, with the generalized heuristic there are many new factors to consider adding for the scoring, such as difficulty values for each column, the weight of individual moves, difficulty scores for different column combinations, and the current positions of markers. Glenn & Aloi used a genetic algorithm to optimize these factors' scoring values, involving

encoding different parts of the game to be run for over 250,000 simulated games. They found their genetic algorithm when ran against the pure Rule of 28 in toy versions, showing an improvement over the Rule of 28. From this improved toy Can't Stop strategy, we can see that there is possible improvement over the Rule of 28, which could translate into the full game.

Another similar game to Can't Stop is backgammon, a very popular gambling dice game with a body of research on developing different optimized strategies and computer algorithms. In backgammon, two players are racing their pieces to their home spot. If a piece is alone at a stage, the opponent can "blot" it, sending back to the start, resetting your progress. Because of the dice rolling, race to finish first, and possibility to lose progress, backgammon strategies and their development process can provide insight into researching Can't Stop. One backgammon method with successful application to Can't Stop is the rollout method, which resulted in a Can't Stop program that wins most games (Canakci, Serrano, Roy, & Kruskal 2016). A rollout is a Monte Carlo evaluation of a position in which the computer plays a position to completion many times (typically thousands of trials), using different random dice sequences in each trial (Tesauro 2002). The rollout method determines the best move by playing the game thousands of times against itself using each option and then picks the option with the best win rate. While this approach wins, it wins via complex computations which are difficult to fully integrate into a heuristic strategy that can be implemented by a human. However, we can analyze the computer play to gain insight into new approaches. The other approach they used was neural networks, a machine learning method. They chose to use a self-teaching network similar to the backgammon network TDGammon. Due to the long training period required to get desirable results, this method was ultimately abandoned despite showing some promise. If we train a computer to play Can't Stop, then we can try to gain insight into new approaches to a heuristic strategy.

Machine learning is the study of artificial intelligence focused on the development of statistical algorithms that learn from data to make accurate inferences about new unseen

data. Neural networks are a machine learning model that takes inputs, passes it through neurons to transform the inputs, and then an activation function outputs a prediction. Neural networks work well for the type of training and learning required for board games. A useful type of neural networks is the reinforcement learning deep Q network. Deep Q networks use Q learning, defined as an algorithm that trains an agent to assign values to all possible actions in its current state, and picks the action with the highest value. Learning to play Can't Stop with a Deep Q Network (Cheong 2020) shows that the deep Q networks can effectively handle the task of learning Can't Stop. Their implementation showed some success in creating a unique strategy. However, it got stuck on a safe, risk-free approach. The safe, risk-free approach is typically less effective than the Rule of 28, making it not an optimal strategy. This is similar to an overly cautious poker player that seeks to minimize risk rather than maximizing their expected value.

There is still further analysis not yet covered using these deep Q networks on Can't Stop that could provide insight on how to improve our current knowledge of the best heuristic strategies. We are going to train a deep Q network to play Can't Stop and perform an analysis on the network to find possible improvements to our understanding of optimal Can't Stop strategies.

3 Methods

For our Can't Stop computer program, we choose to implement a model using deep Q networks (DQN). A DQN consists of a neural network and Q-learning. The combination of the neural network model and Q-learning algorithm is a powerful tool for computer learning.

3.1 Q-Learning

Q-learning is a reinforcement algorithm based around using a Bellman equation as a value iteration update. We define Q-learning by the equation:

$$Q^{new}(S_t, A_t) \leftarrow (1-\alpha) \cdot Q(S_t, A_t) + \alpha \cdot \left(R_{t+1} + \gamma \cdot \underset{a}{max} Q(S_t, a) \right)$$

(Watkins 1989), where the the learned action-value function, Q, directly approximates q_*, the optimal action-value function, independent of the policy being followed. The policy still has an effect in that it determines which state–action pairs are visited and updated (Sutton and Barto 2015). Watkins showed that since this algorithm process is a finite Markov decision process, Q-learning will converge, making this a mathematically powerful algorithm to use.

$Q(S_t, A_t)$ represents the Q-value, or quality, of an action A_t in a state S_t. Q-values are continuous real numbers. Learning rate or step size, α, determines how much new information overrides old information. The values of the learning rate are $0 \leq \alpha \leq 1$, where $\alpha = 0$ is using other prior knowledge (not learning) and $\alpha = 1$ using only recent knowledge. The parameter γ is the discount factor, this determines the importance of the future reward. A value of $\gamma = 0$ will only consider current rewards (R_{t+1}) while values approaching $\gamma = 1$ will go for long-term higher rewards. R_{t+1} is the reward received when moving to next state S_{t+1} after choosing action A_t in state S_t. $\underset{a}{max} Q(S_t, a)$ is the maximum Q-value for S_{t+1}, portraying the best possible reward achievable in S_{t+1}. $Q(S_t, A_t)$ is weighted by one minus learning rate, R_{t+1} is weighted by learning rate, and $\underset{a}{max} Q(S_t, a)$ is weighted by learning rate and discount factor.

As an example of simple Q-learning, consider a 5 by 5 grid maze with a start and an exit. The algorithm's goal is to get to the exit of the maze. There are only 4 possible actions, up, down, left, and right. Here there are 25 states for each of the cells and a terminal state for reaching the exit. For each state, there is a Q-value for each action. Since each state has four Q-values, with 25 states there are 100 different Q-values. We can create a table of Q-values, with each column as an action and each row as a state. At the start of the algorithm, each Q-value in the table is zero. As the algorithm explores the maze, it updates the Q-values using the above equation. We assign a reward value to reaching the exit, so after multiple episodes, reaching the exit, the algorithm will have learned the shortest path to the exit where the best action in each state will have the highest Q-value.

3.2 Neural Network

The second component of DQN is the neural network (NN), also referred to as deep learning. A NN is a statistical model that takes an input vector of variables and builds a nonlinear function $f(X)$ to predict the response Y (James, Witten, Hastie, & Tibshirani 2023). The structure of NN feeds forward through its three layers: input, hidden, and output. The input layer output feeds into each hidden layer unit, which then feeds into the output, returning a prediction. We use an activation function in the hidden layer to introduce nonlinearity to the inputs. A useful type of NN is a convolutional neural network (CNN). A CNN takes inputs that can be represented in a grid-like format, such as an image or game board, and captures spatial patterns located in these grids. For a board game, the prediction can provide insight into possible strategies and optimization.

Using CNN's ability to interpret a board game as an input and extract patterns, we can feed these patterns into a Q-learning algorithm to create Q-values for the large number of state-action pairs. The process of using a CNN is to take a complex input and transform it to be interpreted by the Q-learning algorithm for training a computer to do a task, such as playing Can't Stop.

3.3 Hardware and Software

The training and testing was coded in Python 3.12.10, using the IDE Visual Studio Code 1.110.1. The DQN was built using the Tensorflow 2.20.0 library and Keras 3.13.0 API. The hardware used was a NVIDIA GeForce GTX 1650 GPU, AMD Ryzen 7 5800HS CPU @ 3.20 GHz, and 16GB of RAM.

3.4 Strategies

In order to prevent overfitting while training the DQN and to test its accuracy, we define multiple different strategies, or playstyles, as opponents for the DQN to train against. These are:

- A completely random player
- A cautious player with two levels of skill
- An aggressive player with two levels of skill
- A Rule of 28 player with two levels of skill

An implementation of Glenn & Aloi's genetic algorithm was considered but was determined to be beyond the scope of this thesis.

In order to make some of the strategies more robust, we used the statistical odds for different column combinations using calculations provided by Tate and Renner (See Appendix). The second to last column is the percentage chance of advancing on one roll for your marked columns. The last column is the maximum rolls you can take before your expected gains are exceeded by your expected losses. For example, looking at the 3 column odds section if we have columns 3, 7, and 8 marked each individual roll has a 89% to not bust (we have a 89% chance to get a dice combination with at least one pair that sums to 3, 7, or 8), and we can roll 8 times before we expect the chance of busting to be greater than not busting. The number of times we can roll, X, follows a geometric distribution, where the success, p,

is probability of the individual roll busting. Note that $1 - p$ is the probability a roll not busting, the same as the second to last column. The geometric distribution has the expected value $E(X) = \frac{1-p}{p}$. Taking the expected value, we get the number of rolls we can take before we expect to bust. For example when $p = 11\%$, $\frac{1-0.11}{0.11} = 8.09$ which was rounded down to 8.

3.4.1 Random Player

A player that makes every decision randomly. It will serve as a performance baseline for if the DQN is actually learning to play the game. We will expect that any reasonable human strategy or any successful machine learning strategy will outperform the random player in a majority of games.

3.4.2 Cautious Player

A player that uses a cautious strategy, always stopping after 3 successful rolls or if a column is completed. There are two skill levels, the decent and advanced. The decent level will stop early if the chance of failing a roll is $\geq 75\%$, and will pick columns with existing progress or columns closer to the center of the board. The failing roll chance is based on Tate & Renner's odds table. The advanced level inherits the stopping logic of the decent level and uses the opponent's and its own marker positions to determine which columns to play. This is calculated as a penalty score for each column, where the column with the lowest score is the best choice. The closer an opponent is to completing a column, the higher the penalty score it gets. Owned columns with good progress get a lower penalty score. Each column gets a penalty score that increases the further from the center the column is. The code for calculating this is outlined in the algorithm below:

Algorithm 1 Calculate a penalty score for each column

```
Require: column score = 0
  for each column do
      if a column is completed then
          +1000 column score
      end if
      for each opponent do
          if column progress > 70% then
              +50 column score
          else if column progress > 50% then
              +20 column score
          else if column progress > 30% then
              +5 column score
          end if
      end for
      if We have column progress then
          +(−20) × (column progress) column score
      end if
      +|column − 7| × 0.5 column score  ▷ Higher penalty score the further from the center
  of board
  end for
  return column score
```

3.4.3 Aggressive Player

A player that uses an aggressive strategy, always rolling at least 3 times. There are two skill levels, the decent and advanced. The decent level will stop if a column is completed, the chance of failing $\geq 75\%$, or after rolling the maximum number of times before expected loss exceeds expected gains. The failing roll chance and maximum roll amount is based on Tate & Renner's odds table. The advanced level inherits the stopping logic of the decent level and uses Algorithm 1 for penalty scoring logic to pick columns.

3.4.4 Rule of 28 Player

A player that uses the Rule of 28 strategy, always stopping after reaching the point threshold of 28. The Rule of 28 is defined below:

Table 1: Rule of 28 Scoring (Keller 1986)

Columns	2	3	4	5	6	7	8	9	10	11	12
Spaces	3	5	7	9	11	13	11	9	7	5	3
Value when marked	12	10	8	6	4	2	4	6	8	10	12
Value when advanced	6	5	4	3	2	1	2	3	4	5	6

Add the values for each number marked or advanced in each column during the current turn. Also note that each column counts double when a marker is placed. Add two points if all three columns marked are odd; subtract two points if all three columns are even. Add four points if all three columns are less than eight, or if all three are greater than six. When the total for the current turn reaches 28 or more, stop rolling (but do not stop before all three markers are placed) (Keller 1986).

An example of a turn would look like:

Roll 2,2,3,3. Mark and advance 5. Count 6 (mark 5) plus 3 (advance 5) for a total of $6+3=9$. Roll 1,2,2,4. Mark 3 and 6. Add 10 (mark 3) plus 4 (mark 6) for $9+10+4=23$, plus and additional 4 (because 3, 5, and 6 are all less than 7) for a total of $23+4=27$. Roll 2,4,6,6. Advance 6. Add 2 for a total of $27+2=29$, and then stop since total ≥ 28.
The rule will have you stop within one roll of the correct spot about 75% of the time.

There are two skill levels, the pure and smart. The pure level follows the rule exactly, which Keller warns against, saying to use your own judgement on when to stop in certain cases. The smart level implements judgement on when to stop, not following the rule exactly. This includes stopping if a column is completed before 28 points, and uses Algorithm 1 for penalty scoring logic to pick columns.

3.5 DQN Structure

Since our DQN follows the NN structure, it consists of an input(s), hidden layer(s), and output layer. We also use a policy which will return an action to the environment. This action will generate a reward that will reinforce good decisions. The environment will then

observe the new state S_{t+1}, and is numerically translated into the input layer. The input layer is composed of 51 neurons, each representing a feature the network will consider when determining to stop. These features are the network's permanent progress per column (11 neurons), opponent's permanent progress per column (11 neurons), current temp markers location (11 neurons), completed columns (11 neurons), number of rolls this turn (1 neurons), rule of 28 points (1 neuron), success probability (1 neuron), number of temp columns (1 neuron), number of completed columns (1 neuron), number of opponent completed columns (1 neuron), and number of column completed that turn (1 neuron). Our hidden layer has two layers, with 64 neurons each. The hidden layer size was chosen to maximize learning accuracy, reduce overfitting, and keep computational time low. Each hidden layer having 64 neurons is standard practice in machine learning since it is computationally efficient. Each one uses the Rectified Linear Unit (ReLU) activation function, also known as the ramp function:

$$\text{ReLU}(x) = max(0, x) = \frac{x + |x|}{2} = \begin{cases} x & \text{if } x > 0 \\ 0 & \text{if } x \leq 0 \end{cases}$$

where x is the input to a hidden layer neuron. If x is negative or zero it will return 0, and if x is positive it returns x. Each hidden layer uses 20% dropout to reduce overfitting. The output layer uses a linear activation function to return a Q-value since they are continuous.

See Appendix B for the code used to build the model, as well as the Github repository containing all the code used.

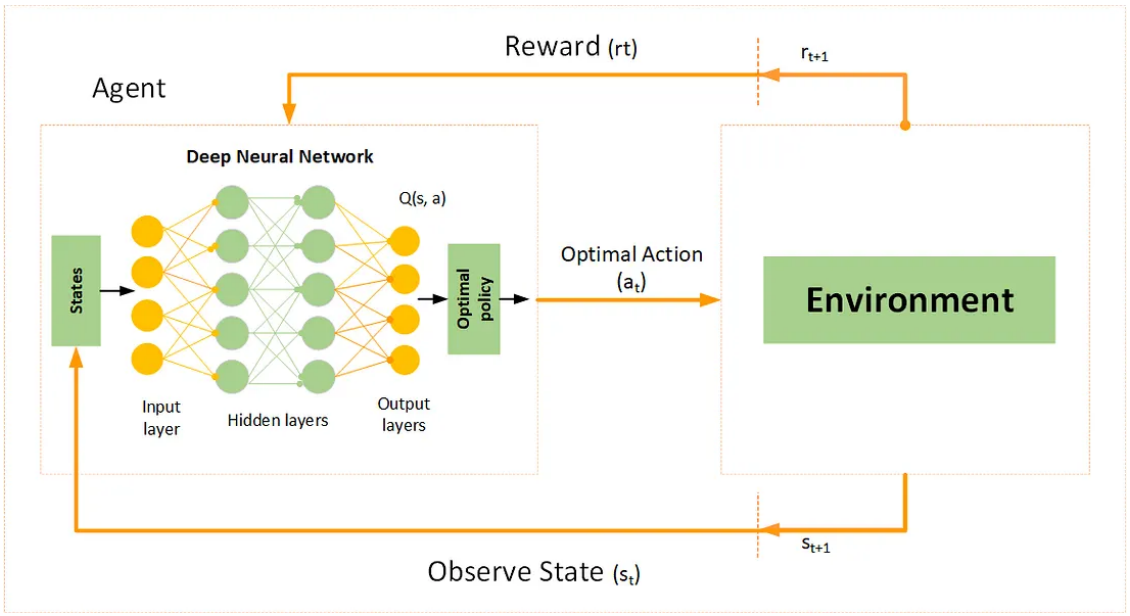

Figure 2: DQN Model Architecture

[2]

3.6 Training

The hyperparameters that we set for the first training session are the learning rate (α), discount factor (γ), and epsilon (ϵ) along with its decay and minimum, and batch size. During our training we will be using the epsilon-greedy policy, where ϵ is the probability of a random action being selected and $1 - \epsilon$ is the probability of the highest Q-value action being selected.

The random action chance starts at 1 and will decrease by a factor of 0.995 every episode until it reaches a minimum of 0.01. The learning rate is initialized to 0.0001, as a sufficiently small learning rate enables the convergence of a Q-value. Discount factor is initialized to 0.95 to help ensure convergence and the long-term reward maximizing strategy while keeping the computational time low. After each training session we will adjust the hyperparameters based on the model's performance.

[2]Figure 2 Retrieved from `https://medium.com/@samina.amin/deep-q-learning-dqn-71c109586bae`

Batch size is the number of random samples from the experience replay buffer that will be used to update the network's Q-values. It is initialized to 16, as it provides low computational time while maintaining good accuracy from during training. The experience replay buffer is the space where each turn is stored, that is the collection of a state, the action, its reward, and the next state. We set this to have a maximum size of 1000, so 16 random samples will be selected from here to update the Q-values.

During the training period, the DQN is trained against the decent and advanced level players, as well as the random player. One episode will represent a single game, or simulation. The opponent is selected randomly for each simulation. For each episode, the overall win rate, win rate against each individual opponent, epsilon value, reward, Q-value, Q-value loss, and average rule of 28 points are recorded to a CSV (comma-separated values) file for analysis.

This analysis will be looking at these recorded values to find insights into possible strategies. This includes looking at the players the DQN has the best and worst win rates against to infer where its strengths and weaknesses lie. Looking at the reward value over time we will analyze the point at which it begins to make choices in an attempt to maximize the reward. Analyzing the overall win rate will inform us of the accuracy of the model, being able to see when the accuracy plateaus. Along with accuracy we can see the mean squared error for the Q-values, giving us more insight into the model's performance.

4 Analysis and Results

4.1 Introduction

We trained on 367,029 simulations split between 18 different training sessions, with a different number of games per training session. The simulations were segmented due to time and hardware constraints. Before each run, the hyperparameters were adjusted based on the previous run's results and the target number of simulations. The goal when tuning the hyperparameters was to guide the algorithm towards our research objective of creating a successful Can't Stop computer algorithm. The hyperparameters that had the most changes are the discount factor and epsilon minimum and decay. Discount factor controls the model's predicted Q-values and therefore the play style, where decreasing it reduces predicted Q-values. Epsilon decay determines the rate at which our model will transition from random actions to strategic actions, and the minimum is the smallest chance for the model to make random decisions that we are satisfied with. The raw data is available for download from the Github repository linked in Appendix B.

4.2 Predicted vs Actual Q-values

We will start evaluating the model's performance by comparing the predicted Q-values to the rewards (actual Q-values) across each episode. Due to the high volume of data points, we smooth the data using the moving average technique with 500 games to reduce the noise in the data. This makes the visualization of the data easier to see and interpret.

Table 2: Average Predicted and Actual Q-values

Run	Avg Reward	Avg Q-value
1	0.3795799	32.111841
2	1.2657258	65.580151
3	1.2009576	63.633910
4	1.0417931	67.565296
5	1.3025428	65.427201
6	0.3447525	41.702243
7	0.9572993	51.280556
8	0.5917869	52.834164
9	0.4691523	60.608044
10	1.6260889	80.384007
11	1.5660336	74.550757
12	0.8143716	4.835290
13	0.4656085	4.294338
14	1.1434122	7.277142
15	1.0963587	6.723077
16	1.0406109	7.017006
17	1.0840146	4.967744
18	1.1788660	5.358426

Figure 3: Runs 1-3 Rewards vs Predicted Q-values (smoothed)

Figure 4: Runs 4-6 Rewards vs Predicted Q-values (smoothed)

Figure 5: Runs 7-9 Rewards vs Predicted Q-values (smoothed)

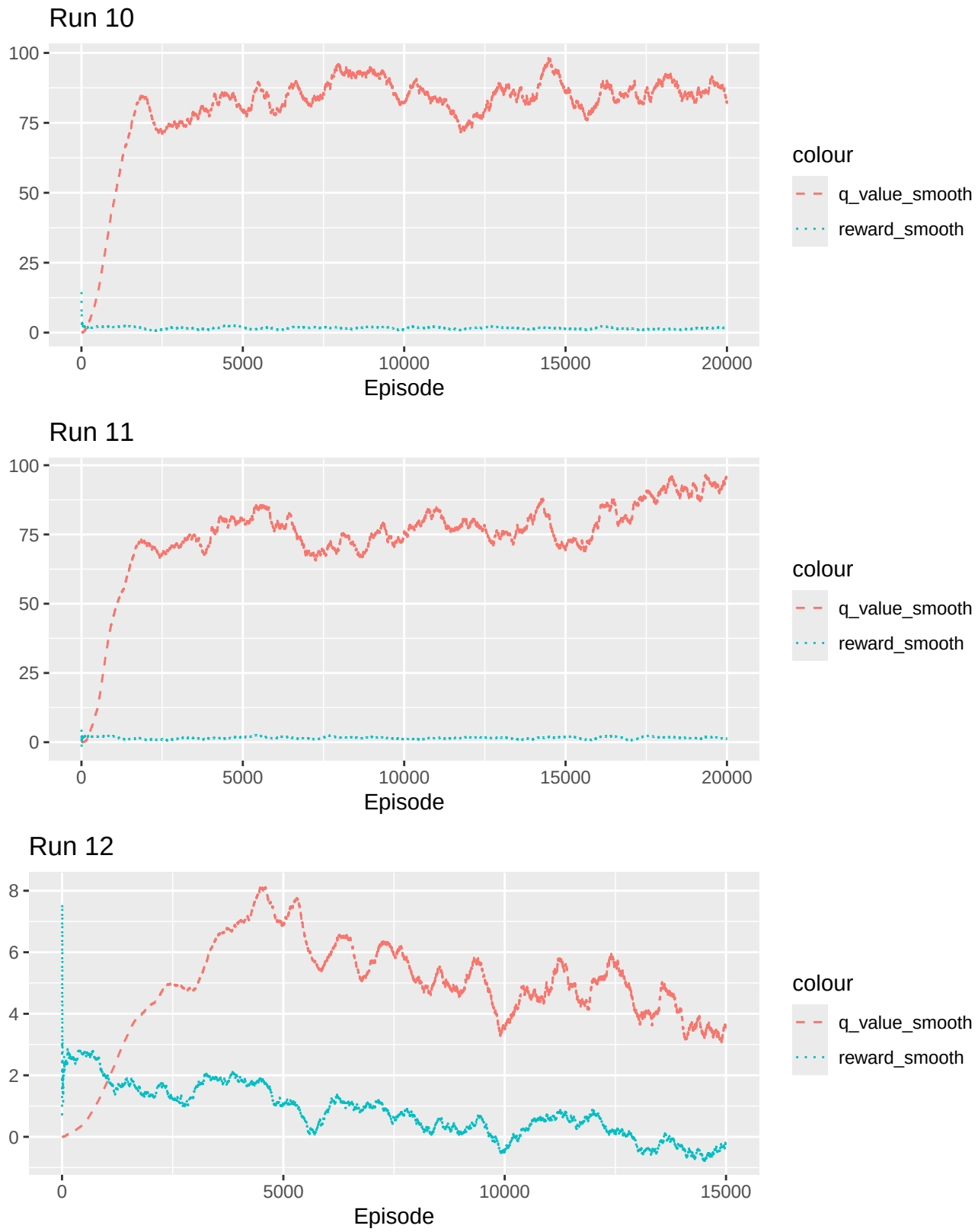

Figure 6: Runs 10-12 Rewards vs Predicted Q-values (smoothed)

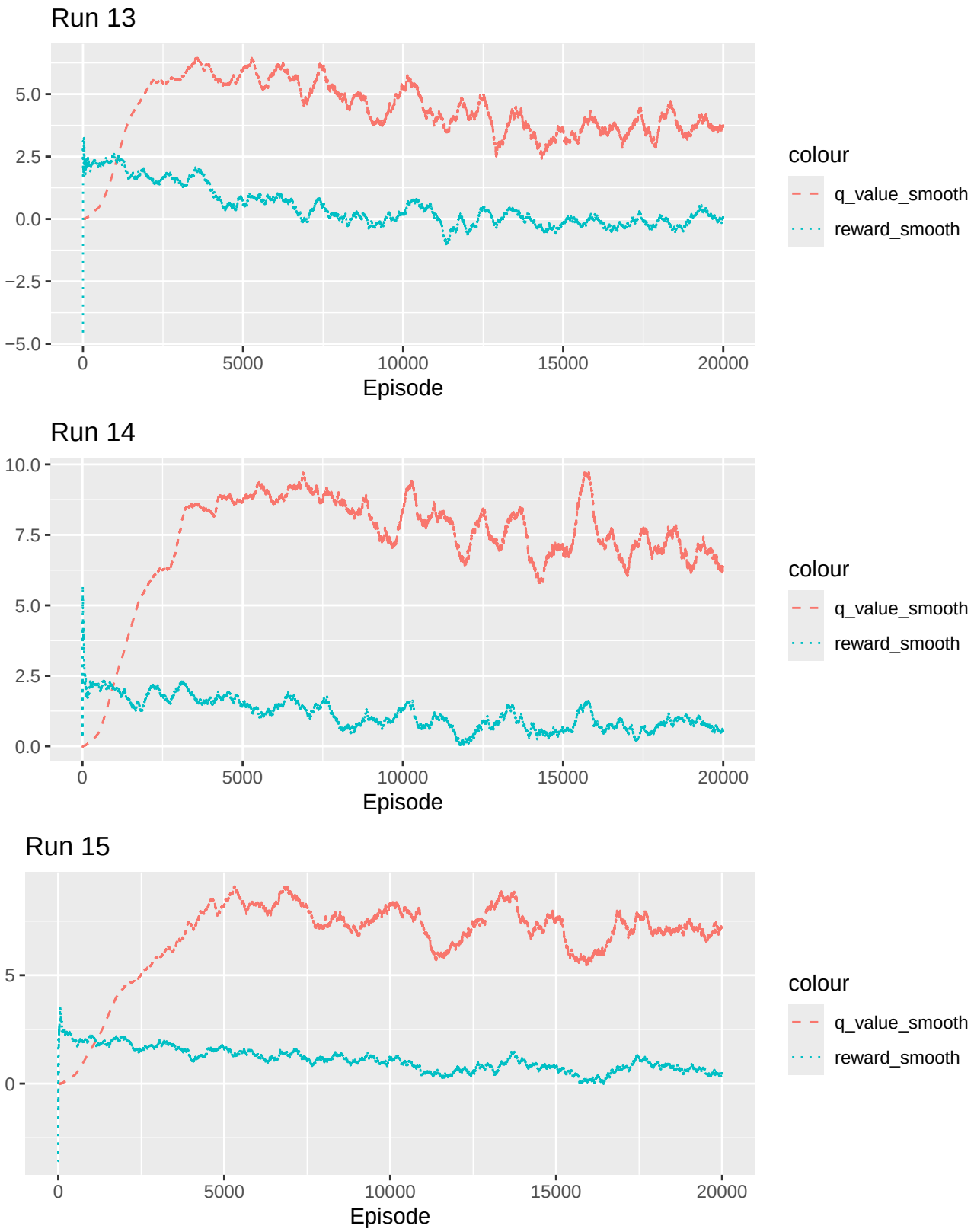

Figure 7: Runs 13-15 Rewards vs Predicted Q-values (smoothed)

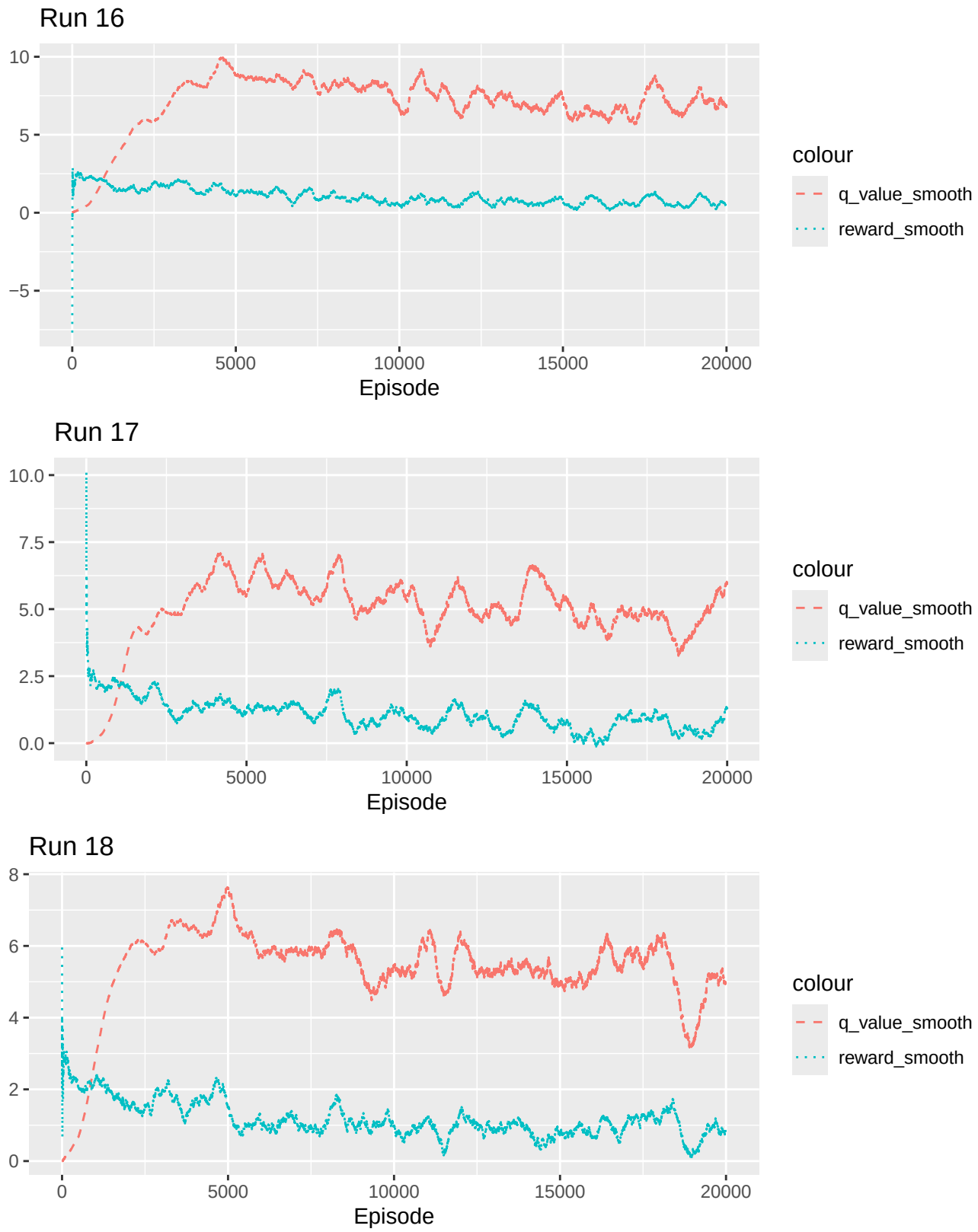

Figure 8: Runs 16-18 Rewards vs Predicted Q-values (smoothed)

4.2.1 Observations and Analysis

Figures 3-8 contain results for the eighteen runs. We can see an overall issue with the model overestimating the Q-values during each training session. While the Q-values are increasing over the training period, the reward remains near zero, not matching the Q-values growth. This means the model is overestimating the quality of its choices each game.

Table 3: Parameter Values

Run	Simulation Size	Learning Rate	Discount Factor	Epsilon Decay	Epsilon Minimum
1	10000	0.0001	0.95	0.99500	0.01
2	20000	0.0001	0.95	0.99990	0.01
3	25000	0.0001	0.95	0.99990	0.01
4	40000	0.0001	0.95	0.99990	0.01
5	25000	0.0001	0.95	0.99990	0.01
6	25000	0.0001	0.95	0.99500	0.01
7	20000	0.0001	0.95	0.99955	0.01
8	10000	0.0001	0.95	0.99500	0.01
9	20000	0.0001	0.95	0.99950	0.01
10	20000	0.0001	0.95	0.99950	0.30
11	20000	0.0001	0.95	0.99950	0.30
12	15000	0.0001	0.50	0.99950	0.10
13	20000	0.0001	0.50	0.99950	0.10
14	20000	0.0001	0.50	0.99950	0.30
15	20000	0.0001	0.50	0.99950	0.30
16	20000	0.0001	0.50	0.99950	0.30
17	20000	0.0001	0.30	0.99950	0.30
18	20000	0.0001	0.30	0.99950	0.30

The first run was 10,000 simulations, with $\alpha = 0.0001$, $\gamma = 0.95$, and ϵ decay and minimum set to 0.995 and 0.01 respectively. We can see the overestimation immediately, which could be due to the beginning stages of training where the model is playing the game for the very first time. For the second and third run, the number of simulations was increased to 20,000 and 25,000 respectively. Since the training periods will be longer the ϵ decay is changed to 0.9999 for a slower decay which will allow the model to spend more time exploring new strategies. There is a small increase in the reward value across the training period, implying the model is slowly learning.

The fourth training session was 40,000 simulations with the same hyperparameter values as the previous run. The length of the run was increased in an attempt to reduce the overestimating issue by giving the model a longer training session. As seen in Figure 4 above, increasing the simulations did not have a noticeable effect in reducing the Q-value overestimation. The fifth and sixth training sessions were both 25,000 simulations. The sixth session was cut short at 22,029 games due to the training not finishing in the amount of time available. The training taking too long is possibly due to the decrease of the ϵ decay to 0.995 causes the model to spend more time exploring. The reward values during the fifth session are the highest overall with an average of 1.302. The sixth session's ϵ decay was set to 0.995 to see if a shorter exploration period would affect the overestimation. While the predicted value peak was lower than previous runs, the reward values were still very low.

For the seventh training session, the number of simulations was reduced to 20,000 and the ϵ decay was set to 0.99955. For the eighth training run, the number of simulations was again reduced to 10,000 and ϵ decay was set to 0.995. The peak of the predicted values was lower, hanging around 60. For the ninth training, the simulations were returned to 20,000 and ϵ decay was set to 0.9995. Interestingly the model reached the plateau of its predicted Q-values sooner than any other training session.

The tenth and eleventh training session had similar results to run 9. The ϵ minimum was raised to 0.3 to reduce the model falling into overestimating bad strategies. However, session 10 and 11 had an average reward of 1.626 and 1.566 respectively. This implies the gap between the predicted and actual values is getting smaller. This could be because enough training time has passed for the model to start learning the game. For the twelfth training session the simulation number was reduced to 15,000 and ϵ minimum was set to 0.1, and most significantly the discount factor was reduced to 0.5 to encourage short term reward seeking behavior. As seen in Figure 6, there was a larger decrease in the overestimation, with the predicted values getting closer to the actual values as the model finished training.

The next three training sessions are run with almost the same parameters as session 12

to see if the decreased discount will improve the win rate. The ϵ minimum was set to 0.3 for 14 and 15 to encourage more learning exploration. As seen in Figure 7 and Table 2, the overestimation continues to be reduced. The average reward values increase with the predicted values in sessions 14 and 15. We can see that reducing the discount factor did reduce the overestimation as predicted. The model making short term decisions appears to be increasing the quality of the model's training.

Session 16's rewards are similar to session 15 with slightly larger predicted values. For session 17, the discount factor was reduced to 0.3 because of the lack of improvement from session 15 to 16 to see if an even smaller discount factor would decrease the difference between the reward and predicted values. This did cause session 17 to have lower overall values, and when the reward and predicted values increase in session 18 the difference between the values increases as well.

4.2.2 Conclusion

Looking at the predicted vs actual Q-values to determine our models performance, we can see that it was not as successful as we had hoped with the high discount factor. There was a large presence of overestimation in the predicted values, which seemed to slow the model's training process. However, upon reducing the discount factor we saw an immediate reduction in overestimation. This reduction remains present through the rest of the training. Starting training with the model prioritizing long term rewards can cause the model to chase larger Q-values that are unattainable. The model learned to make decisions that it incorrectly predicted will lead to high rewards while in practice it was making poor decisions. This leads to the large difference seen between the predictions and actual values, the model learned to overestimate the quality of its decisions. Since a Can't Stop game is on the shorter side, typically 8-10 turns per player, the higher discount factor is trying to find rewards for a large number of turns that likely won't be reached. Starting the training process with a decreased discount factor and increasing it as the model performance increases could lead to a better

overall performance of the model in a shorter training period. Different small discount factor values could be experimented with to see which one best fits the Can't Stop play style.

4.3 Opponent Win Rates

Looking at the model's win rate, we will analyze how its performance changes across each training session. Also, we look at how it performs against the individual strategies to find the models weaknesses and strengths.

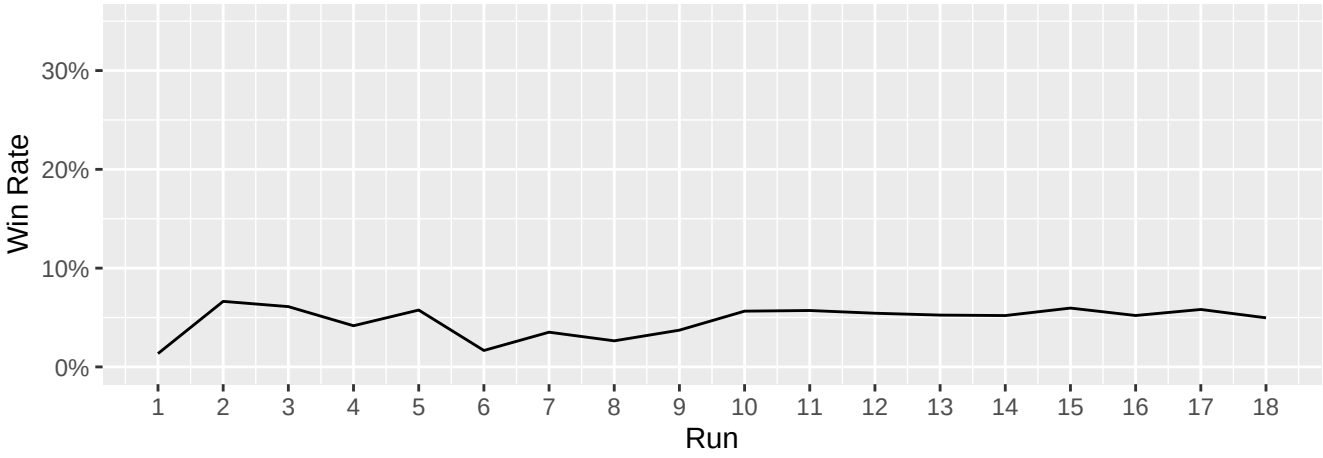

Figure 9: Overall Win Rate per Training Session

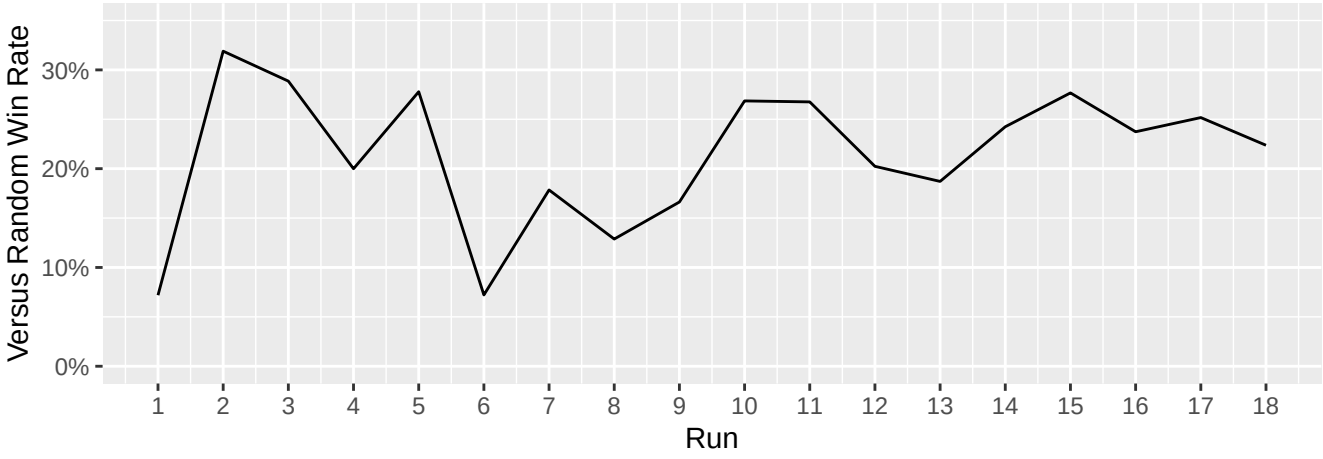

Figure 10: Versus Random Win Rate per Training Session

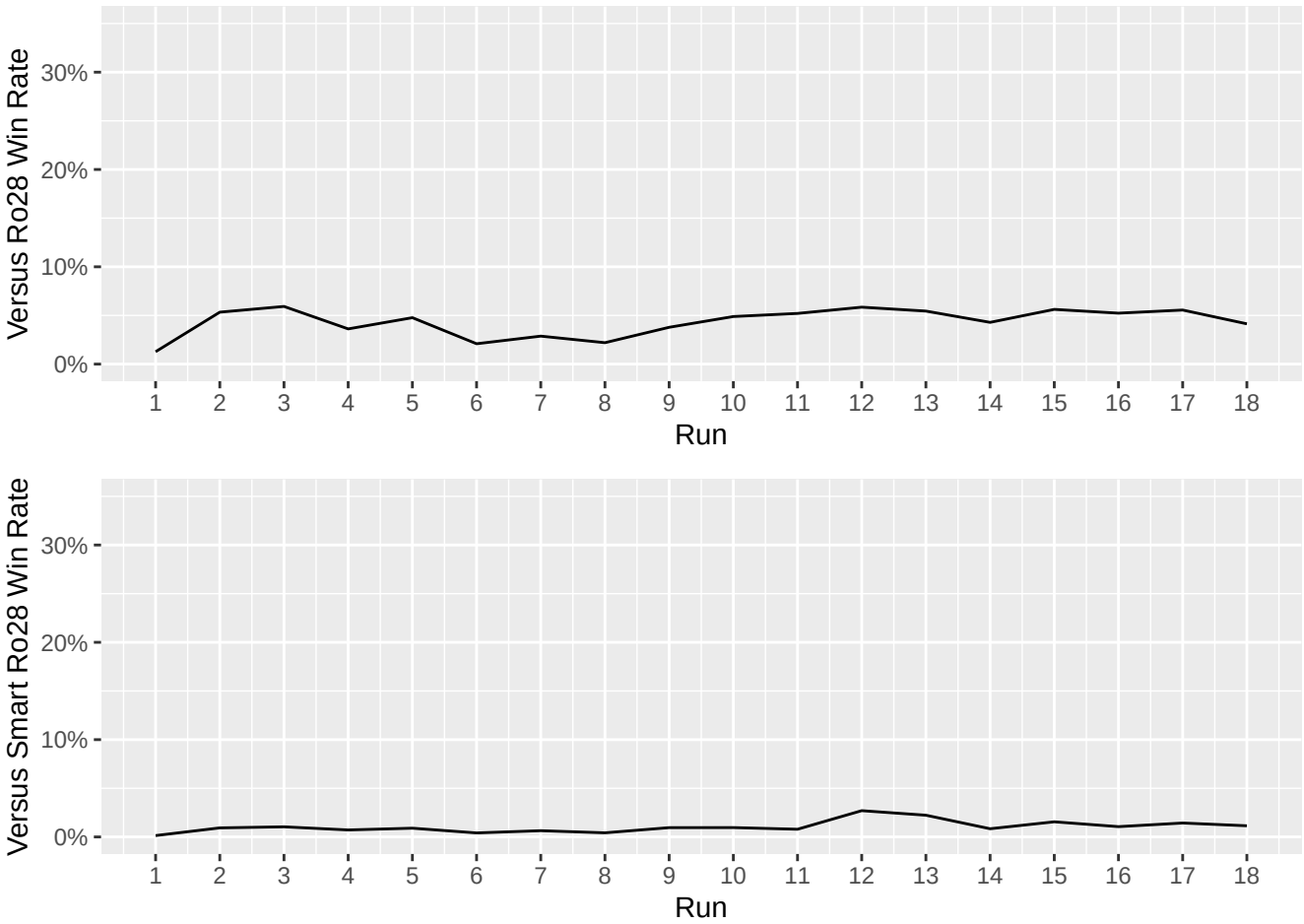

Figure 11: Versus Ro28 and Smart Ro28 Win Rate per Training Session

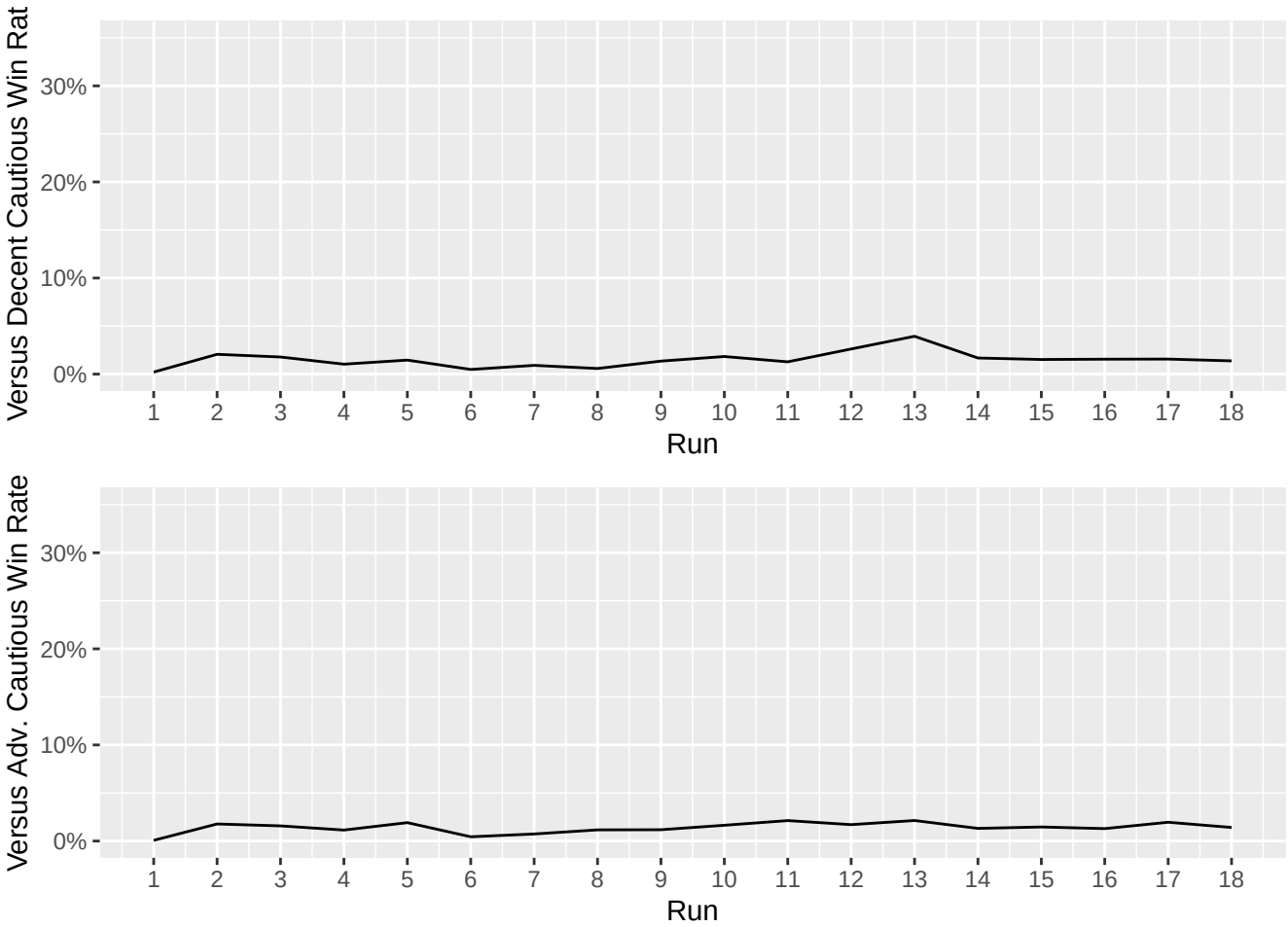

Figure 12: Versus Cautious Players Win Rate per Training Session

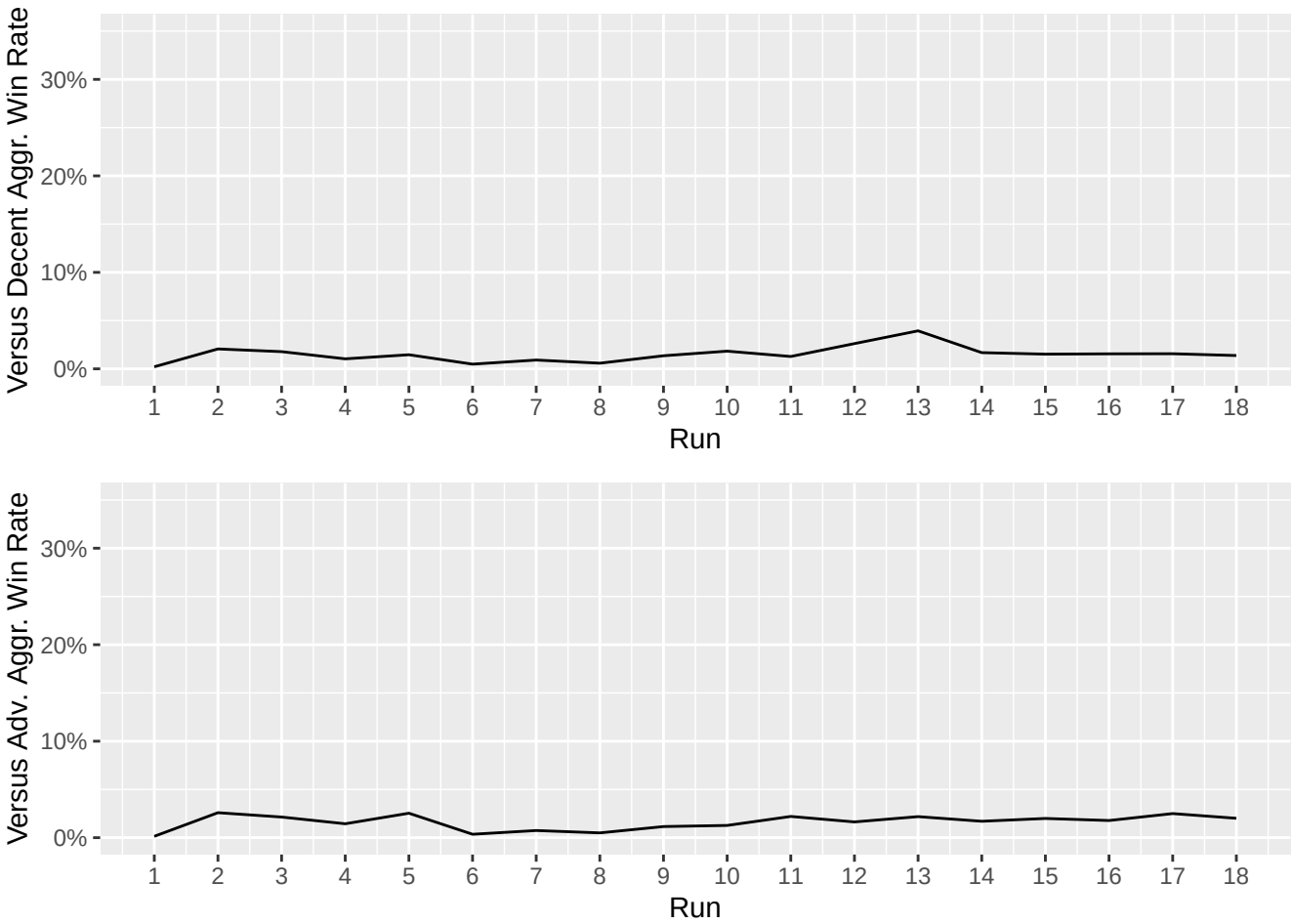

Figure 13: Versus Aggressive Players Win Rate per Training Session

4.3.1 Observations and Analysis

As shown in Figure 9, the win rate increases from training session 1 to 2, as we would expect and want to see. It then fluctuates before noticeably dropping at training session 6. The win rate decrease was likely a result of changing the ϵ decay to 0.995, which caused the model to quickly stop exploring new strategies during training. For the following training sessions the win rate slightly fluctuates and then increases into the end of the training. Although the discount factor was decreased for training session 12, compared to 10 and 11 it does not appear to have made a noticeable impact on the win rate. For sessions 13 and 14, the win rate does not increase staying the same 5.2%. Session 15 has a minor increase

to almost 6%. The continued training with the decreased discount factor likely has led to this win rate increase. However, for session 16 the win rate drops down to before, similar to 13 and 14. Session 17 the discount was decreased further, which had a positive effect on the win rate, looking like session 15. However, going into session 18 with the same parameters the win rate dropped. This could be a natural fluctuation in the training process.

Using the random player as a baseline for the model's performance, we hope to see the model be able to perform better than random actions. However, as seen in Figure 10 it did not perform as well as we hoped. However, compared to the other individual player win rates the model had the highest win rate against the random player for every training session. The increases and decreases follow the overall win rate trend. The second training had the highest win rate out of the 12 training sessions. The win rate seems to be reacting to the change in ϵ decay. When the decay is changed to be faster in a longer training session, we see a major decrease of 27% to 7%. This is supported by session 10, where the win rate is above 20% for the first time since session 5. As stated previously, the ϵ minimum was decreased to 0.3, which has a similar effect to slowing the decay. This keeps the model in the exploration stage longer. In session 12 the impact of the decreased discount factor is more visible, the decrease forces the model to make more short term choices so we expect a slight performance decrease. While session 13 dips a little, we see a big rise through session 14 and 15. Session 16 drops to a similar win rate as session 14. From this drop the discount factor is decreased to 0.3 to improve win rates, which we see in session 17's increase. Session 18's win rate drops, not continuing session 17's increase.

The Rule of 28 is one the best performing strategies in practice. Our model's performance is seen in Figure 11. The model's performance against the pure Ro28 player was the second highest individual win rate during the training. The win rate follows the same overall trend as seen earlier, a rise in the beginning that falls in session 6 followed by a final rise. Comparing the win rate to the smart Ro28, we see a noticeable decrease in performance. Since one of the models inputs was its Ro28 score, this possibly gave it an advantage over the pure

strategy. However, introducing more advanced decisions put our model at a disadvantage. Though the win rate against the smart Ro28 stays at $\approx 1\%$, in the last training session we see an increase to almost 3%. This could correlate with the decrease in the discount factor, the model being able to make more short sighted decisions gives it an advantage over the strategy's complex decision-making algorithm. For the pure Ro28, sessions 13, 15, 16, and 17 all have similar win rates to 12, implying the discount factor decrease is responsible for improving model performance. Session 14 has a slight dip, which is likely an outlier due to a lower number of matches against the Ro28 player. Against the smart Ro28 player, the performance of session 13 follows 12 but it dips back down for sessions 14 though 18.

Figure 12 tells us the model did poorly against both levels of the cautious strategies. The model did slightly better against the decent level than the advanced level, but not by much. For session 13 against the decent level we see a spike, with the highest win rate of $\approx 4\%$. Looking ahead to Figure 13, the outcome of the model's performance against the aggressive players follows a similar trend of low performance with a spike at session 13 against the decent level. The spike at session 13 could be due to a good luck streak for the model or a low amount of matches. This suggests our model's weakness were these more complex decision-focused strategies. The factors these strategies consider like opponent positioning put them at a greater advantage.

4.3.2 Conclusion

Looking at the win rates overall and individually, the model did not perform as well as we expected. Despite that we can still gain insight into how the model was developing its own strategy. The model was being held back by a high discount factor, which made it difficult to hone an effective strategy. While giving the model training time with a smaller discount factor did not show the larger growth we desired, the consistency of its performance was much better. More training with a smaller discount factor that gradually increases with the models performance would likely result in a more powerful model. Another alternative is

training a new model on a low and slowly increasing discount factor to ensure it only learns short term reward behavior to start. As well, more time in the early training stage with a high ϵ would allow the model to explore and find more unique strategies. The model appeared to have been developing a working strategy, as it was a noticeable amount of games against the random player. The only non-random player the model showed some success against was pure Ro28. Similar to the pure Ro28, the models weakness was the complex decision making which included consideration of opponents progress. This implies the model's strategy was similar to Ro28, sharing a similar weakness.

4.4 Feature Importance

For testing the importance of each feature, we use the permutation feature importance method. This method tests the model's features by randomizing a feature's values, then recording the model's win rate. We will use the win rate against the Rule of 28 player as our baseline to compare when testing the randomized feature values. For each feature, they will be permuted for 200 games with the win rate being recorded. The baseline win rate against the Rule of 28 player was 5.5%, so the change in win rate percentage is measured as the difference between the baseline and permuted win rate. The results are shown in Table 4, ordered from most to least important. The features with a negative percent change indicates the model performed better with the feature permuted compared to the base line.

Table 4: Permutation Feature Importance

Ranking	Feature	Permutated Win Rate	Win Rate % Change
1	column_completed_this_turn	0.010	4.5
2	temp_marker_col_7	0.015	4.0
3	opp_progress_col_7	0.025	3.0
4	my_completed_count	0.025	3.0
5	opp_progress_col_5	0.030	2.5
6	completed_col_9	0.030	2.5
7	opp_completed_count	0.030	2.5
8	temp_marker_col_8	0.035	2.0

9	completed_col_2	0.035	2.0
10	completed_col_11	0.035	2.0
11	opp_progress_col_8	0.040	1.5
12	temp_marker_col_10	0.040	1.5
13	completed_col_4	0.040	1.5
14	completed_col_5	0.040	1.5
15	num_temp_markers	0.040	1.5
16	my_progress_col_12	0.045	1.0
17	completed_col_10	0.045	1.0
18	completed_col_12	0.045	1.0
19	success_probability	0.045	1.0
20	opp_progress_col_10	0.050	0.5
21	temp_marker_col_5	0.050	0.5
22	my_progress_col_4	0.055	0.0
23	opp_progress_col_11	0.055	0.0
24	rolls_this_turn	0.055	0.0
25	opp_progress_col_4	0.060	-0.5
26	completed_col_6	0.060	-0.5
27	temp_marker_col_3	0.065	-1.0
28	temp_marker_col_4	0.065	-1.0
29	completed_col_7	0.065	-1.0
30	completed_col_8	0.065	-1.0
31	my_progress_col_2	0.070	-1.5
32	my_progress_col_3	0.070	-1.5
33	my_progress_col_7	0.070	-1.5
34	my_progress_col_8	0.070	-1.5
35	my_progress_col_10	0.070	-1.5
36	opp_progress_col_3	0.070	-1.5
37	opp_progress_col_12	0.070	-1.5
38	temp_marker_col_2	0.070	-1.5
39	temp_marker_col_9	0.070	-1.5
40	temp_marker_col_11	0.070	-1.5
41	temp_marker_col_12	0.070	-1.5
42	completed_col_3	0.070	-1.5
43	my_progress_col_6	0.075	-2.0
44	my_progress_col_11	0.075	-2.0
45	opp_progress_col_2	0.080	-2.5

46	opp_progress_col_9	0.085	-3.0
47	my_progress_col_5	0.090	-3.5
48	opp_progress_col_6	0.090	-3.5
49	temp_marker_col_6	0.090	-3.5
50	ro28_points	0.105	-5.0
51	my_progress_col_9	0.110	-5.5

4.4.1 Observations and Analysis

Looking at the table, we see that columns_completed_this_turn, temp_marker_column_7, opp_progress_column_7, and my_completed_count showed the highest decrease in performance when permuted. We expect columns_completed_this_turn feature to be important since having correct information about your progress during your turn is an essential part of the game. The model was likely stopping upon completing a column to secure progress, a very common strategy which is usually a good choice. The second most important feature, temp_marker_column_7, implies the model had realized that column 7 was the most likely to be rolled and focused on winning that column. This is supported by the next important feature opp_progress_column_7, which is the model gauging if they are going to lose column 7 to the opponent. The model has learned to prioritize winning column 7 before the opponent. The my_completed_count feature means the model was making decisions based on how close it was to winning, and we can see in with the 7th ranked opp_completed_count that the model learned to change its decision making based on how close it was to winning or losing. Looking at the features with a negative percent change, my_progress_col_9 had the biggest difference at -5.5. This possibly implies that the model could have learned a bad strategy involving column 9. Randomizing the value could have caused the model to play better by accident or the feature was redundant and a different feature was already capturing the trend of column 9. Another possibility could be related to my_progress_col_5, which also had a very low score. It's possible the model learned that these two columns were not worth going for, as they both have equal probability of being rolled. If we compare the

probability of getting 5 or 9 to their column height, it could be a disadvantage to try and win these columns. This pattern is not seen with any other symmetric columns, so it could be unique to 5 and 9. Interestingly the Rule of 28 points feature also recorded a 5% win rate increase when permuted. The model likely learned a bad understanding of the Rule of 28, so randomizing the values helped it perform better essentially ignoring it. As well since it was going against the actual Rule of 28 player, it was more successful not using the feature it did not understand against a strategy that did.

4.4.2 Conclusion

From the feature importance analysis, we can see the model developed both helpful and unhelpful strategies during its training. As we would expect, the model developed a strategy involving using the columns completed during a turn and completed columns overall. As well the model was using the opponent's completed column count, implying the model had learned to change the play style based on how close it or the opponent was to winning. This is a very helpful play style that likely was behind the winning games during training. As well the model learned to differentiate the value between columns, as it prioritizes column 7 over all the others. Focusing on these features in future training might improve the models performance. This most likely came naturally from the fact 7 has the highest probability of being rolled. The Rule of 28 points feature's lack of importance implies the model might perform better if trained without the feature, instead it might be better to add later into the training process. Many features had little to no impact on the performance, so reducing the features to be less redundant could improve the performance in future training.

5 Conclusion

We trained a Deep Q-Network to learn and play Can't Stop. However, the DQN was not successful at playing the game well as it failed to meet the performance baseline. We expected the DQN to win a majority of games against the random player, however we saw a 21.4% average win rate. There are many possibilities as to why the model failed to meet the performance baseline, such as poor hyperparameter tuning, an over complicated training environment, or using an ineffective learning algorithm.

The main problem seen during the training process was overestimation of the reward values. This was caused by setting the discount factor too high, forcing the model to chase and overestimate long term rewards unachievable in a typical game. While this was adjusted for the last 3rd of the training, as seen in Figures 6-8, the win rate did not experience enough of an impact. This is likely because the amount of time spent training with the parameter set too high could not be offset with the amount of total training time left. The model spent majority of its time trying to do a futile task, and this likely led to the model creating bad connections with its features. The majority of the model's core knowledge is learned in the initial training stages, so spending that crucial training time learning a very ineffective strategy was a significant hindrance on the model's performance.

The training environment we created involved the model playing games against 7 different playstyles at random to prevent the model from overfitting a hyperspecific strategy. This possibly had the opposite effect as the strategy the model developed was not successful against any playstyle. Training against one or two approaches and the baseline could create a better performing model, with the other playstyles being used for testing only to evaluate the model. Along with a difficult learning environment, reducing the number of features could also improve the model's performance. Since many of the features had no impact or a negative impact on the performance, reducing the features could allow the model to concentrate on a smaller set of features. Another alternative is to use an ensemble learning technique, either bagging or boosting. From James, Witten, Hastie, and Tibshirani (2021),

bagging is taking many subsets of the training data (or features), and build separate models for each set, and combine the outputs to make a prediction. Boosting is similar except the separate models are added sequentially into the main model. Both methods reduce the variance and overfitting bias in the model.

With time, the model could be retrained using a smaller discount factor that increases with performance. Since Can't Stop is a shorter game, models could benefit from a smaller discount factor. Another possible parameter change for future training is increasing the learning rate, which was not able to be tested during this analysis. Increasing the step-size could reduce training time and improve performance as the model uses new information more quickly. Along with these changes to the parameters, reducing the scope of the training environment with less playstyles would be tested. Giving the model a simpler environment to train against could be similar in the approach of studying the toy versions of games to find strategies. The overestimation seen in this analysis is a common issue with training DQN models, so a recent solution is the Double Q-learning algorithm proposed by Van Hasselt, Guez, and Silver (2016). This works by giving the model two Q-tables, one for q-values of the best action choice and the other to evaluate the q-value of the action from the first table. This implementation reduces overestimation however is more computationally expensive compared to standard Q-learning. As discussed above, using the bagging method could also help the model develop the features better. With more time and computing power, future work using the double Q-learning method with bagging, reducing the training environment, and better parameterization could produce a better performing model with a unique strategy to analyze and learn from.

References

Canakci, B., Serrano, S., Roy, O., and Kruskal, C. (2016), "The Can't Stop Game", REU-CAAR 2016, Available at https://www.cs.umd.edu/~gasarch/REU/reupapers.html.

Cheong, Y. J. (2020), "Learning to play Can't Stop with a Deep Q Network", Author. Available at https://yujia21.github.io/personal/cant-stop.html.

Fang, H., Gleen, J., and Kruskal, C. (2008), "Retrograde Approximation Algorithms for Jeopardy Stochastic Games", *International Computer Games Association*, 31(2), 77-96. Available at https://doi.org/10.3233/icg-2008-31203.

Glenn, J. R., & Aloi, C. J. (2009), "A Generalized Heuristic for Can't Stop", *Proceedings of the Twenty-Second International FLAIRS Conference*, 421-426. Available at https://aaai.org/papers/flairs-2009-123/.

James, G., Witten, D., Hastie, T., and Tibshirani, R. (2021), *An Introduction to Statistical Learning* (2nd ed.), New York City, NY: Springer.

Keller, M. (1986), "Can't Stop? Try The Rule of 28", Author, Available at https://www.solitairelaboratory.com/cantstop.html.

Packel, E. (2006), *The Mathematics of Games and Gambling* (2nd ed.), Washington, DC: The Mathematical Association of America.

Rapoport, A. (1966), *Two Person Game Theory*, Garden City, NY: Dover Publications.

Sutton, R., and Barto, A. (2015), *Reinforcement Learning: An Introduction* (2nd ed.), Cambridge, MA: The MIT Press.

Tate and Renner (2007), "Can't Stop Odds", Author, Available at https://www.taterenner.com/cantstop.php.

Tesauro, G. (2002), "Programming backgammon using self-teaching neural nets", *Artificial Intelligence*, 134, 181-199, Available at https://doi.org/10.1016/S0004-3702(01)00110-2.

Van Hasselt, H., Guez, A., and Silver, D. (2016), "Deep reinforcement learning with double q-learning", *Proceedings of the AAAI conference on artificial intelligence*, 30(1), Available

at https://doi.org/10.1609/aaai.v30i1.10295.

Watkins, C. (1989), *Learning from Delayed Rewards*, Ph.D. dissertation, King's College London.

A Appendix: Odds Table

Can't Stop Odds

Single Column Odds

2	13%	0
3	23%	0
4	36%	0
5	45%	1
6	56%	1
7	64%	2
8	56%	1
9	45%	1
10	36%	0
11	23%	0
12	13%	0

Two Column Odds

2	3	32%	0
2	4	44%	1
2	5	53%	1
2	6	63%	2
2	7	71%	2
2	8	67%	2
2	9	56%	1
2	10	47%	1
2	11	36%	1
2	12	26%	0
3	4	47%	1
3	5	53%	1
3	6	64%	2
3	7	71%	2
3	8	68%	2
3	9	64%	2
3	10	56%	1
3	11	45%	1
3	12	36%	1
4	5	61%	2
4	6	72%	3
4	7	77%	3
4	8	75%	3
4	9	68%	2
4	10	67%	2
4	11	56%	1
4	12	47%	1
5	6	73%	3
5	7	78%	4
5	8	77%	3
5	9	69%	2
5	10	68%	2
5	11	64%	2
5	12	56%	1
6	7	84%	5
6	8	82%	5
6	9	77%	3
6	10	75%	3
6	11	68%	2
6	12	67%	2
7	8	84%	5
7	9	78%	4
7	10	77%	3
7	11	71%	2
7	12	71%	2
8	9	73%	3
8	10	72%	3
8	11	64%	2
8	12	63%	2
9	10	61%	2
9	11	53%	1
9	12	53%	1
10	11	47%	1
10	12	44%	1
11	12	32%	0

Three Column Odds

2	3	4	52%	1
2	3	5	58%	1
2	3	6	68%	2
2	3	7	75%	3
2	3	8	76%	3
2	3	9	71%	2
2	3	10	63%	2
2	3	11	53%	1
2	3	12	44%	1
2	4	5	66%	2
2	4	6	76%	3
2	4	7	81%	4
2	4	8	82%	5
2	4	9	76%	3
2	4	10	74%	3
2	4	11	63%	2
2	4	12	55%	1
2	5	6	77%	3
2	5	7	81%	4
2	5	8	83%	5
2	5	9	76%	3
2	5	10	76%	3
2	5	11	71%	2
2	5	12	63%	2
2	6	7	86%	6
2	6	8	88%	7
2	6	9	83%	5
2	6	10	81%	4
2	6	11	76%	3
2	6	12	74%	3
2	7	8	89%	8
2	7	9	84%	5
2	7	10	83%	5
2	7	11	78%	4
2	7	12	78%	4
2	8	9	82%	5
2	8	10	82%	5
2	8	11	74%	3
2	8	12	74%	3
2	9	10	71%	2

From www.taterenner.com

Figure 14: Can't Stop Odds (Tate and Renner 2007)

2	9	11	64%	2
2	9	12	63%	2
2	10	11	58%	1
2	10	12	55%	1
2	11	12	44%	1
3	4	5	67%	2
3	4	6	74%	3
3	4	7	79%	4
3	4	8	80%	4
3	4	9	78%	4
3	4	10	76%	3
3	4	11	66%	2
3	4	12	58%	1
3	5	6	77%	3
3	5	7	79%	4
3	5	8	81%	4
3	5	9	78%	4
3	5	10	76%	3
3	5	11	71%	2
3	5	12	64%	2
3	6	7	86%	6
3	6	8	85%	6
3	6	9	83%	5
3	6	10	82%	5
3	6	11	76%	3
3	6	12	74%	3
3	7	8	89%	8
3	7	9	84%	5
3	7	10	84%	5
3	7	11	78%	4
3	7	12	78%	4
3	8	9	84%	5
3	8	10	83%	5
3	8	11	76%	3
3	8	12	76%	3
3	9	10	78%	4
3	9	11	71%	2
3	9	12	71%	2
3	10	11	66%	2
3	10	12	63%	2
3	11	12	53%	1
4	5	6	80%	4
4	5	7	85%	6
4	5	8	85%	6
4	5	9	80%	4
4	5	10	82%	5
4	5	11	78%	4
4	5	12	71%	2
4	6	7	89%	8
4	6	8	91%	10
4	6	9	86%	6
4	6	10	88%	7
4	6	11	83%	5
4	6	12	82%	5
4	7	8	90%	9
4	7	9	89%	8
4	7	10	88%	7
4	7	11	84%	5
4	7	12	83%	5
4	8	9	86%	6
4	8	10	88%	7
4	8	11	82%	5
4	8	12	81%	4
4	9	10	82%	5
4	9	11	76%	3
4	9	12	76%	3
4	10	11	76%	3
4	10	12	74%	3
4	11	12	63%	2
5	6	7	89%	8
5	6	8	90%	9
5	6	9	87%	7
5	6	10	86%	6
5	6	11	84%	5
5	6	12	82%	5
5	7	8	91%	10
5	7	9	85%	6
5	7	10	89%	8
5	7	11	84%	5
5	7	12	84%	5
5	8	9	87%	7
5	8	10	86%	6
5	8	11	83%	5
5	8	12	83%	5
5	9	10	80%	4
5	9	11	78%	4
5	9	12	76%	3
5	10	11	78%	4
5	10	12	76%	3
5	11	12	71%	2
6	7	8	92%	12
6	7	9	91%	10
6	7	10	90%	9
6	7	11	89%	8
6	7	12	89%	8
6	8	9	90%	9
6	8	10	91%	10
6	8	11	85%	6
6	8	12	88%	7
6	9	10	85%	6
6	9	11	81%	4
6	9	12	83%	5
6	10	11	80%	4
6	10	12	82%	5
6	11	12	76%	3
7	8	9	89%	8
7	8	10	89%	8
7	8	11	86%	6
7	8	12	86%	6
7	9	10	85%	6
7	9	11	79%	4
7	9	12	81%	4
7	10	11	79%	4
7	10	12	81%	4
7	11	12	75%	3
8	9	10	80%	4
8	9	11	77%	3
8	9	12	77%	3
8	10	11	74%	3
8	10	12	76%	3
8	11	12	68%	2
9	10	11	67%	2
9	10	12	66%	2
9	11	12	58%	1
10	11	12	52%	1

From www.taterenner.com

Figure 15: Can't Stop Odds (Tate and Renner 2007)

B Appendix: Python Code & Github

https://github.com/JCameronCarter/HonorsThesis

```
import numpy as np
import random
from collections import deque
import tensorflow as tf
from tensorflow import keras
from keras import layers
from typing import List, Tuple, Set

# Import from existing file
from CantStop import (
    CantStopGame,
    PlayerStrategy,
    RuleOf28,
    AdvancedAggressivePlayer
)

class DQNPlayer(PlayerStrategy):
    # Deep Q-Learning player for Can't Stop

    def __init__(self, state_size=51, action_size=2,
                 load_model=None, training=True):
        self.state_size = state_size
        self.action_size = action_size
        self.training = training
```

```
        # Hyperparameters
        self.learning_rate = 0.0001
        self.gamma = 0.3
        self.epsilon = 1.0
        self.epsilon_min = 0.3
        self.epsilon_decay = 0.9995
        self.batch_size = 16

        # Experience replay memory
        self.memory = deque(maxlen=1000)

        # Metric Tracking
        self.episode_rewards = []
        self.episode_q_values = []
        self.last_loss = 0.0
        self.episode_ro28_points = []

        # Game state tracking
        self.rolls_this_turn = 0
        self.columns_started_this_turn = set()

        # Build networks
        if load_model:
            self.model = keras.models.load_model(load_model)
            self.target_model = keras.models.load_model(load_model)
        else:
```

```
        self.model = self._build_model()
        self.target_model = self._build_model()
        self.update_target_model()

    def _build_model(self):
        #Build the Q-network
        model = keras.Sequential([
            layers.Dense(64, activation='relu',
            input_shape=(self.state_size,)),
            layers.Dropout(0.2),
            layers.Dense(64, activation='relu'),
            layers.Dropout(0.2),
            layers.Dense(self.action_size, activation='linear')
        ])
        model.compile(optimizer=keras.optimizers.
                      Adam(learning_rate=self.learning_rate),loss='mse')
        return model

    def update_target_model(self):
        #Copy weights from model to target_model
        self.target_model.set_weights(self.model.get_weights())

    def _encode_state(self, game: CantStopGame) -> np.ndarray:
        #Convert game state to feature vector
        features = []
        current_player = game.current_player
        opponent_id = 1 - current_player
```

```python
# Model's permanent progress (11 values (1 per column), normalized
for col in range(2, 13):
    progress = game.board.player_markers[current_player][col]
    height = game.board.Column_Heights[col]
    features.append(progress / height)

#Opponent permanent progress (11 values(1 per column), normalized)
for col in range(2, 13):
    progress = game.board.player_markers[opponent_id][col]
    height = game.board.Column_Heights[col]
    features.append(progress / height)

# Current temp markers (11 values (1 per column), normalized)
for col in range(2, 13):
    if col in game.board.temp_markers:
        temp_progress = game.board.temp_markers[col] -
                game.board.player_markers[current_player][col]
        height = game.board.Column_Heights[col]
        features.append(temp_progress / height)
    else:
        features.append(0.0)

# Completed columns (11 binary (1 per column))
for col in range(2, 13):
    features.append(1.0 if col in game.board.completed else 0.0)
```

```
        # Number of rolls this turn (1, normalized)
        features.append(min(self.rolls_this_turn / 10.0, 1.0))

        # Rule of 28 points (1, normalized)
        ro28_points = RuleOf28.calulate_points(game,
                                                self.columns_started_this_turn)
        features.append(ro28_points / 50.0)

        # Success probability (1)
        temp_cols = set(game.board.temp_markers.keys())
        if len(temp_cols) == 3:
            features.append(game.board.
                            get_success_probability(game.board.temp_markers))
        else:
            features.append(1.0)

        # Number of temp markers (1, normalized)
        features.append(len(game.board.temp_markers) / 3.0)

        # Model's completed columns (1)
        my_completed = sum(
            1 for col in game.board.completed
            if game.board.player_markers[current_player][col] >=
                                                game.board.Column_Heights[col]
        )
        features.append(my_completed / 3.0)
```

```
        # Opponent completed columns (1)
        opp_completed = sum(
            1 for col in game.board.completed
            if game.board.player_markers[opponent_id][col] >=
                                                game.board.Column_Heights[col]
        )
        features.append(opp_completed / 3.0)

        # Any column completed this turn? (1)
        completed_this_turn = any(
            game.board.temp_markers.get(col, 0) >=
                                                game.board.Column_Heights[col]
            for col in game.board.temp_markers.keys()
        )
        features.append(1.0 if completed_this_turn else 0.0)

        return np.array(features, dtype=np.float32)

    def remember(self, state, action, reward, next_state, done):
        #Store experience in replay memory
        self.memory.append((state, action, reward, next_state, done))

    def act(self, state):
        #Choose action using epsilon-greedy policy
        if self.training and np.random.rand() <= self.epsilon:
            return random.randrange(self.action_size)
```

```
        state = state.reshape(1, -1)
        q_values = self.model.predict(state, verbose=0)

        if self.training:
            self.episode_q_values.append(np.max(q_values[0]))

        return np.argmax(q_values[0])

    def replay(self):
        #Train on a batch of experiences
        if len(self.memory) < self.batch_size:
            return

        minibatch = random.sample(self.memory, self.batch_size)

        states = np.array([exp[0] for exp in minibatch])
        actions = np.array([exp[1] for exp in minibatch])
        rewards = np.array([exp[2] for exp in minibatch])
        next_states = np.array([exp[3] for exp in minibatch])
        dones = np.array([exp[4] for exp in minibatch])

        # Predict Q-values
        current_q = self.model.predict(states, verbose=0)
        next_q = self.target_model.predict(next_states, verbose=0)

        # Update Q-values using Bellman equation
        for i in range(self.batch_size):
```

```
            if dones[i]:
                current_q[i][actions[i]] = rewards[i]
            else:
                current_q[i][actions[i]] = rewards[i] +
                                            self.gamma * np.max(next_q[i])

        # Train the model
        history =self.model.fit(states, current_q, epochs=1, verbose=0)

        self.last_loss = history.history['loss'][0]

        # Decay epsilon
        if self.epsilon > self.epsilon_min:
            self.epsilon *= self.epsilon_decay

    def reset_episode_metrics(self):
        #Reset metrics at the start of an episode
        self.episode_rewards = []
        self.episode_q_values = []
        self.episode_ro28_points = []

    def get_episode_metrics(self):
        #Get metrics for the current episode
        avg_reward = np.mean(self.episode_rewards)
                                if self.episode_rewards else 0.0
        avg_q_value = np.mean(self.episode_q_values)
                                if self.episode_q_values else 0.0
```

```
        avg_ro28_points = np.mean(self.episode_ro28_points)
                                if self.episode_ro28_points else 0.0
        return {
            'avg_reward': avg_reward,
            'avg_q_value': avg_q_value,
            'loss': self.last_loss,
            'avg_ro28_points': avg_ro28_points
        }

    def decide_to_roll(self, game: CantStopGame) -> bool:
        if self.rolls_this_turn == 0:
            self.columns_started_this_turn = set()
            return True

        state = self._encode_state(game)
        action = self.act(state)

        # Store state for later reward calculation
        self.last_state = state
        self.last_action = action

        return action == 1

    def choose_move(self, game: CantStopGame, valid_moves:
                                List[Tuple[int, int]]) -> Tuple[int, int]:
        self.rolls_this_turn += 1
```

```
        # Use heuristics for move selection
        temp_cols = set(game.board.temp_markers.keys())

        for move in valid_moves:
            if move[0] in temp_cols or move[1] in temp_cols:
                return move

        return min(valid_moves, key=lambda m: abs(m[0] - 7) + abs(m[1] - 7))



class RLTrainer:
    #Training environment for DQN player

    def __init__(self, opponent=None):
        self.dqn_player = DQNPlayer(training=True)
        self.opponent = opponent if opponent else AdvancedAggressivePlayer()
        self.episodes = 0
        self.wins = 0
        self.losses = 0

        self.cumulative_rewards=[]
        self.cumulative_q_values = []
        self.cumulative_ro28_points=[]
        self.cumulative_losses=[]

    def calculate_reward(self, game: CantStopGame, player_id: int,
                         action: int, busted: bool, stopped: bool, won: bool)
```

```
                    -> float:
        #Calculate reward for the current action

        if won:
            return 100.0

        if busted:
            return -50.0

        if stopped:
            temp_sum = sum(game.board.temp_markers.values())
            progress_reward = temp_sum * 2.0

            completed_bonus = sum(
                10.0 for col in game.board.temp_markers.keys()
                if game.board.temp_markers[col] >=
                                        game.board.Column_Heights[col]
            )

            return progress_reward + completed_bonus

        return -0.5

    def train_episode(self, verbose=False):
        #Train for one episode
        game = CantStopGame(num_players=2)
```

```
        self.dqn_player.reset_episode_metrics()

        dqn_is_player_0 = random.random() > 0.5

        if dqn_is_player_0:
            strategies = [self.dqn_player, self.opponent]
        else:
            strategies = [self.opponent, self.dqn_player]

        for strategy in strategies:
            if hasattr(strategy, 'rolls_this_turn'):
                strategy.rolls_this_turn = 0
            if hasattr(strategy, 'columns_started_this_turn'):
                strategy.columns_started_this_turn = set()

        episode_memories = []

        while not game.is_game_over():
            player = game.current_player
            strategy = strategies[player]

            if hasattr(strategy, 'rolls_this_turn'):
                strategy.rolls_this_turn = 0
            if hasattr(strategy, 'columns_started_this_turn'):
                strategy.columns_started_this_turn = set()

            turn_active = True
```

```
        turn_start_state = None

        while turn_active:
            is_dqn_turn = (dqn_is_player_0 and player == 0) or
                                (not dqn_is_player_0 and player == 1)

            if is_dqn_turn and strategy.rolls_this_turn > 0:
                turn_start_state = self.dqn_player._encode_state(game)
                if hasattr(strategy, 'columns_started_this_turn'):
                    ro28_points = RuleOf28.calulate_points(game,
                              strategy.columns_started_this_turn)
                    self.dqn_player.episode_ro28_points.append(
                                                    ro28_points)

            should_roll = strategy.decide_to_roll(game)

            if not should_roll:
                if is_dqn_turn and turn_start_state is not None:
                    next_state = self.dqn_player._encode_state(game)
                    reward = self.calculate_reward(
                                game, player, 0, False, True, False)
                    episode_memories.append((
                      turn_start_state, 0, reward, next_state, False))

                    self.dqn_player.episode_rewards.append(reward)

                game.stop()
```

```
                turn_active = False
                continue

            dice = game.roll_dice()
            valid_moves = game.get_valid_moves(dice)

            if not valid_moves:
                if is_dqn_turn and turn_start_state is not None:
                    next_state = self.dqn_player._encode_state(game)
                    reward = self.calculate_reward(
                                    game, player, 1, True, False, False)
                    episode_memories.append((
                        turn_start_state, 1, reward, next_state, False))

                    self.dqn_player.episode_rewards.append(reward)

                game.bust()
                turn_active = False
                continue

            move = strategy.choose_move(game, valid_moves)

            if hasattr(strategy, 'columns_started_this_turn'):
                for col in move:
                    if col not in game.board.temp_markers:
                        strategy.columns_started_this_turn.add(col)
```

```
            game.make_move(move)

            if is_dqn_turn and turn_start_state is not None:
                next_state = self.dqn_player._encode_state(game)
                reward = self.calculate_reward(
                                    game, player, 1, False, False, False)
                episode_memories.append((
                        turn_start_state, 1, reward, next_state, False))

                self.dqn_player.episode_rewards.append(reward)
                turn_start_state = next_state




    dqn_won = (dqn_is_player_0 and game.winner == 0) or
                            (not dqn_is_player_0 and game.winner == 1)

    if dqn_won:
        self.wins += 1
        final_reward = 100.0
    else:
        self.losses += 1
        final_reward = -100.0

    if episode_memories:
        last_exp = episode_memories[-1]
        episode_memories[-1] =
```

```
            (last_exp[0], last_exp[1], final_reward, last_exp[3], True)
        self.dqn_player.episode_rewards.append(final_reward)
    for exp in episode_memories:
        self.dqn_player.remember(*exp)

    if len(self.dqn_player.memory) >= self.dqn_player.batch_size:
        self.dqn_player.replay()

    metrics = self.dqn_player.get_episode_metrics()
    self.cumulative_rewards.append(metrics['avg_reward'])
    self.cumulative_q_values.append(metrics['avg_q_value'])
    self.cumulative_ro28_points.append(metrics['avg_ro28_points'])
    self.cumulative_losses.append(metrics['loss'])

    self.episodes += 1

    if verbose and self.episodes % 10 == 0:
        win_rate = self.wins / self.episodes if self.episodes>0 else 0
        print(f"Episode{self.episodes},Wins:{self.wins},
        Losses:{self.losses},"f"Win Rate:{win_rate:.2%} Epsilon:
        {self.dqn_player.epsilon:.3f}")

    return dqn_won
```